*Functions are powerful. Try writing some yourself.*

*As before, don't forget to run the setup code below before jumping into question 1.*

```
# SETUP. You don't need to worry for now about what this code does or how it wor
from learntools.core import binder; binder.bind(globals())
from learntools.python.ex2 import *
print('Setup complete.')
```

```
Setup complete.
```

**1. Complete the body of the following function according to its docstring.**

**HINT: Python has a built-in function `round`.**

Ans.

[54]:
```
def round_to_two_places(num):
    """Return the given number rounded to two decimal places.

    >>> round_to_two_places(3.14159)
    3.14
    """
    # Replace this body with your own code.
    # ("pass" is a keyword that does literally nothing. We used it as a placeholder
    # because after we begin a code block, Python requires at least one line of code)
    pass

    return round(num, 2)


# Check your answer
q1.check()
```

Correct

**2. The help for `round` says that `ndigits` (the second argument) may be negative.
What do you think will happen when it is? Try some examples in the following cell.**

Ans.

```
def round_test(num):
    return round(num, -4)

print (round_test(409247027))
```

```
409250000
```

**3. In the previous exercise, the candy-sharing friends Alice, Bob and Carol tried to split candies evenly. For the sake of their friendship, any candies left over would be smashed. For example, if they collectively bring home 91 candies, they'll take 30 each and smash 1.**

**Below is a simple function that will calculate the number of candies to smash for *any* number of total candies.**

**Modify it so that it optionally takes a second argument representing the number of friends the candies are being split between. If no second argument is provided, it should assume 3 friends, as before.**

**Update the docstring to reflect this new behaviour.**

Ans.

```
def to_smash(total_candies, n = 3):
    """Return the number of leftover candies that must be smashed after distributing
    the given number of candies evenly between n-friends, where the default number of friends is

    >>> to_smash(91)
    1

    >>> to_smash(91, 4)
    3
    """
    return total_candies % n
print (to_smash (800, 9))
print (to_smash (800))
q3.check()
```

```
8
2
```

**9. It may not be fun, but reading and understanding error messages will be an important part of your Python career.**

**Each code cell below contains some commented buggy code. For each cell...**

**1. Read the code and predict what you think will happen when it's run.**
**2. Then uncomment the code and run it to see what happens. (**Tip**: In the kernel editor, you can highlight several lines and press `ctrl`+`/` to toggle commenting.)**
**3. Fix the code (so that it accomplishes its intended purpose without throwing an exception)**

```
round_to_two_places(9.9999)
```

[61]:
```
10.0
```

```
smallest_abs = min(abs(x), abs(y))
print(smallest_abs)
```

```
5
```

```
def f(x):
    y = abs(x)
    return y

print(f(5))
```

```
5
```